

Contents

Flutter Complete Study Guide — Bisaya + English (Basic to Pro)	1
PART 1: FOUNDATIONS (Pasiuna)	1
PART 2: PAGSUGOD SA FLUTTER PROJECT	2
PART 3: WIDGETS (Ang Kasingkasing sa Flutter)	3
PART 4: LAYOUT SYSTEM (Detalyado)	5
PART 5: NAVIGATION (Pag-lakat sa Different Screens)	6
PART 6: STATE MANAGEMENT	7
PART 7: PAG-HANDLE UG INPUT (Forms, Buttons, Gestures)	8
PART 8: ASYNCHRONOUS PROGRAMMING UG API CALLS	10
PART 9: LISTS UG DATA DISPLAY	11
PART 10: LOCAL DATA STORAGE	12
PART 11: THEMING UG STYLING	13
PART 12: ANIMATIONS	14
PART 13: TESTING	15
PART 14: ARCHITECTURE PATTERNS (Pro Level)	15
PART 15: PERFORMANCE OPTIMIZATION	16
PART 16: BUILDING UG DEPLOYMENT	17
PART 17: QUICK REFERENCE CHEAT SHEET	17
STUDY ROADMAP	18

Flutter Complete Study Guide — Bisaya + English (Basic to Pro)

PART 1: FOUNDATIONS (Pasiuna)

1.1 Unsa ang Flutter?

Ang Flutter kay usa ka **open-source UI framework** nga gihimo sa Google, gamiton para maghimo ug **cross-platform apps** — usa ra ka codebase, pwede na mo-run sa Android, iOS, Web, Desktop (Windows/Mac/Linux).

- Naga-gamit ug **Dart** nga programming language
- Dili niya gamiton ang native components sa OS — mo-draw siya sa iyang KAUGALINGON nga UI gamit ang **Skia rendering engine** (mao nga parehas gyud ang tan-awon sa Android ug iOS)
- **“Everything is a Widget”** — mao ni ang core nga philosophy sa Flutter

1.2 Ngano Flutter?

Benefit	Ipasabot
Single codebase	Usa ra ka code, daghan platforms
Hot Reload	Makita dayon ang changes nimo nga wala mo-restart sa app
Fast Performance	Compiled directly sa native machine code (dili “bridge” sama sa uban)
Rich Widgets	Daghan na’y built-in nga UI components
Strong Community	Daghan packages sa pub.dev

1.3 Dart Language Basics (kinahanglan una ni bag-o mag-Flutter)

```
void main() {
  print('Hello Flutter!');
}

// Variables
String name = 'Juan';
int age = 20;
double gpa = 3.75;
bool isStudent = true;
var city = 'Cebu'; // type inference
dynamic anything = 'pwede bisan unsa';

// Null safety (importante kaayo sa Dart)
String? nickname; // pwede null
String realName = 'Juan'; // dili pwede null

// Functions
int add(int a, int b) {
  return a + b;
}

// Arrow function (short syntax)
int multiply(int a, int b) => a * b;

// Classes
class Student {
  String name;
  int age;

  Student(this.name, this.age);

  void introduce() {
    print('Ako si $name, $age years old.');
```

PART 2: PAGSUGOD SA FLUTTER PROJECT

2.1 Installation ug Setup

```
flutter doctor      # i-check kung kompleto ang setup nimo
flutter create my_app # maghimo ug bag-ong project
cd my_app
flutter run         # i-run ang app
```

2.2 Project Structure

```
my_app/
├── lib/
```

	└─ main.dart	-- diri nimo isulat ang app code (starting point)
	└─ android/	-- native Android files
	└─ ios/	-- native iOS files
	└─ pubspec.yaml	-- listahan sa dependencies/packages
	└─ test/	-- unit tests

2.3 Basic App Structure

```
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'My First App',
      home: Scaffold(
        appBar: AppBar(title: Text('Hello Flutter')),
        body: Center(
          child: Text('Welcome!'),
        ),
      ),
    );
  }
}
```

Explanation: ang main() mao ang entry point (sugod) sa app. Ang runApp() mao ang mag-launch sa widget nga imong gi-build. Ang MaterialApp mao ang naghatag ug Material Design theme sa tibuok app.

PART 3: WIDGETS (Ang Kasingkasing sa Flutter)

3.1 Unsa ang Widget?

Sa Flutter, **LITERAL ANG TANAN WIDGET** — bisan text, button, layout, padding, color — widget tanan. Nag-combine-combine ka lang ug widgets para mahimo'g UI.

3.2 StatelessWidget vs StatefulWidget

```
// StatelessWidget – dili mag-usab ang iyang data/state (static ra)
class Greeting extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Text('Hello, walay mausab ni!');
  }
}

// StatefulWidget – pwede mag-usab ang data/state (dynamic)
```

```

class Counter extends StatefulWidget {
  @override
  _CounterState createState() => _CounterState();
}

class _CounterState extends State<Counter> {
  int count = 0;

  void increment() {
    setState(() {
      count++; // mag-usab ang value, ug mo-rebuild ang UI automatic
    });
  }

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        Text('Count: $count'),
        ElevatedButton(
          onPressed: increment,
          child: Text('Add'),
        ),
      ],
    );
  }
}

```

Importante gyud ni: ang `setState()` mao ang naga-tell sa Flutter nga “naa’y na-usab, i-rebuild ang UI.” Kung wala nimo gigamit ang `setState()`, dili mo-update ang screen bisan ba nag-usab imong data.

3.3 Common Widgets Cheat Sheet

Widget	Gamit
<code>Text()</code>	Pag-display ug text
<code>Container()</code>	Box nga pwede i-style (padding, margin, color, size)
<code>Row()</code>	I-arrange ang widgets HORIZONTAL
<code>Column()</code>	I-arrange ang widgets VERTICAL
<code>Stack()</code>	I-overlap ang widgets (usa sa taas sa lain)
<code>ListView()</code>	Scrollable nga listahan
<code>Image()</code>	Pag-display ug picture
<code>Icon()</code>	Pag-display ug icon
<code>ElevatedButton()</code>	Button nga naa’y shadow/elevation
<code>TextField()</code>	Input field para sa text
<code>Scaffold()</code>	Basic structure sa screen (app bar, body, drawer, atbp.)
<code>AppBar()</code>	Ang bar sa taas sa screen
<code>Padding()</code>	Naghatag ug space palibot sa widget
<code>Center()</code>	I-center ang widget
<code>SizedBox()</code>	Naghatag ug fixed nga size o space (spacer)

3.4 Layout Example

```
Container(  
  padding: EdgeInsets.all(16),  
  color: Colors.blue,  
  child: Column(  
    mainAxisAlignment: MainAxisAlignment.center,  
    crossAxisAlignment: CrossAxisAlignment.start,  
    children: [  
      Text('Ngalan: Juan'),  
      SizedBox(height: 10), // spacer  
      Text('Course: BSCS'),  
      Row(  
        children: [  
          Icon(Icons.star),  
          Text('4.5 rating'),  
        ],  
      ),  
    ],  
  ),  
)
```

Explanation: `mainAxisAlignment` naga-control sa alignment sa “main direction” (vertical para sa `Column`), samtang `crossAxisAlignment` naga-control sa “cross direction” (horizontal para sa `Column`).

PART 4: LAYOUT SYSTEM (Detalyado)

4.1 Row ug Column

```
Row(  
  mainAxisAlignment: MainAxisAlignment.spaceBetween,  
  children: [  
    Icon(Icons.home),  
    Icon(Icons.search),  
    Icon(Icons.person),  
  ],  
)
```

4.2 Expanded ug Flexible

```
Row(  
  children: [  
    Expanded(  
      flex: 2,  
      child: Container(color: Colors.red, height: 50),  
    ),  
    Expanded(  
      flex: 1,  
      child: Container(color: Colors.blue, height: 50),  
    ),  
  ],  
)
```

```
],
)
```

Explanation: ang Expanded naga-fill sa available space. Ang flex naga-determine sa ratio — sa example, ang red 2x ka dako compared sa blue.

4.3 Stack (Overlapping Widgets)

```
Stack(
  children: [
    Image.network('https://example.com/photo.jpg'),
    Positioned(
      bottom: 10,
      right: 10,
      child: Icon(Icons.favorite, color: Colors.red),
    ),
  ],
)
```

4.4 GridView

```
GridView.count(
  crossAxisCount: 2, // 2 columns
  children: List.generate(10, (index) {
    return Container(
      color: Colors.teal,
      child: Center(child: Text('Item $index')),
    );
  }),
)
```

PART 5: NAVIGATION (Pag-lakat sa Different Screens)

5.1 Basic Navigation

```
// Pag-adto sa bag-ong screen
Navigator.push(
  context,
  MaterialPageRoute(builder: (context) => SecondScreen()),
);

// Pag-balik sa nag-una nga screen
Navigator.pop(context);
```

5.2 Named Routes

```
MaterialApp(
  initialRoute: '/',
  routes: {
```

```

        '/': (context) => HomeScreen(),
        '/details': (context) => DetailsScreen(),
        '/profile': (context) => ProfileScreen(),
    },
)

// Pag-navigate gamit ang route name
Navigator.pushNamed(context, '/details');

```

5.3 Pag-pasa ug Data Between Screens

```

// Pag-send ug data
Navigator.push(
    context,
    MaterialPageRoute(
        builder: (context) => DetailsScreen(studentName: 'Juan'),
    ),
);

// Pag-receive sa laing screen
class DetailsScreen extends StatelessWidget {
    final String studentName;
    DetailsScreen({required this.studentName});

    @override
    Widget build(BuildContext context) {
        return Scaffold(
            body: Text('Hello, $studentName'),
        );
    }
}

```

PART 6: STATE MANAGEMENT

6.1 Ngano Importante ang State Management?

Sa gagmay nga apps, ang `setState()` sakto ra. Pero sa dagko nga apps (daghan screens naga-share ug data), lisod na i-manage kung `setState()` ra imong gamiton — mao nga naa’y mga “state management solutions.”

6.2 `setState()` (Basic, built-in)

Gigamit na sa naunang example — para lang sa simple/local nga state sa usa ka widget.

6.3 Provider (Pinakakasagaran gamiton, beginner-friendly)

```

// 1. Himo ug ChangeNotifier class
class CounterProvider extends ChangeNotifier {
    int count = 0;
}

```

```

void increment() {
    count++;
    notifyListeners(); // mag-tell sa tanang naga-"listen" nga naa'y update
}
}

// 2. I-wrap ang app sa ChangeNotifierProvider
void main() {
    runApp(
        ChangeNotifierProvider(
            create: (context) => CounterProvider(),
            child: MyApp(),
        ),
    );
}

// 3. I-access/gamiton bisan asa sa app
Consumer<CounterProvider>(
    builder: (context, counterProvider, child) {
        return Text('Count: ${counterProvider.count}');
    },
)

// Pag-call sa function
context.read<CounterProvider>().increment();

```

6.4 Uban pang State Management Options

Option	Level	Note
setState()	Beginner	Built-in, simple lang, local state
Provider	Beginner-Intermediate	Pinakakasagaran gamiton sa community
Riverpod	Intermediate	Mas modern nga version sa Provider
Bloc/Cubit	Advanced	Structured, maayo para sa dagko nga apps
GetX	Intermediate	All-in-one (state, navigation, dependency injection)

PART 7: PAG-HANDLE UG INPUT (Forms, Buttons, Gestures)

7.1 TextField ug Forms

```

class LoginForm extends StatefulWidget {
    @override
    _LoginFormState createState() => _LoginFormState();
}

```



```

class _LoginFormState extends State<LoginForm> {
  final TextEditingController emailController = TextEditingController();

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        TextField(
          controller: emailController,
          decoration: InputDecoration(labelText: 'Email'),
        ),
        ElevatedButton(
          onPressed: () {
            print('Email: ${emailController.text}');
          },
          child: Text('Submit'),
        ),
      ],
    );
  }
}

```

7.2 Form Validation

```

final _formKey = GlobalKey<FormState>();

Form(
  key: _formKey,
  child: Column(
    children: [
      TextFormField(
        validator: (value) {
          if (value == null || value.isEmpty) {
            return 'Kinahanglan sulti-an ang email';
          }
          return null;
        },
      ),
      ElevatedButton(
        onPressed: () {
          if (_formKey.currentState!.validate()) {
            print('Valid ang form!');
          }
        },
        child: Text('Submit'),
      ),
    ],
  ),
)

```

7.3 GestureDetector (Custom Taps/Swipes)

```
GestureDetector(  
  onTap: () => print('Nag-tap!'),  
  onDoubleTap: () => print('Double tap!'),  
  onLongPress: () => print('Gi-hold!'),  
  child: Container(  
    color: Colors.orange,  
    child: Text('Tap ko'),  
  ),  
)
```

PART 8: ASYNCHRONOUS PROGRAMMING UG API CALLS

8.1 Future ug async/await

```
Future<String> fetchData() async {  
  await Future.delayed(Duration(seconds: 2)); // mag-simulate ug delay  
  return 'Data na-kuha!';  
}  
  
void loadData() async {  
  print('Nagsugod...');  
  String result = await fetchData();  
  print(result);  
}
```

Explanation: ang Future naga-represent sa value nga makuha ra later (dili dayon). Ang async naga-mark sa function nga naa'y asynchronous nga operations sulod, ug ang await naga-pause sa execution hangtod ma-resolve ang Future.

8.2 HTTP Requests (Pag-call ug API)

```
# sa pubspec.yaml, dugangan una ang dependency  
dependencies:  
  http: ^1.1.0
```

```
import 'package:http/http.dart' as http;  
import 'dart:convert';  
  
Future<void> fetchStudents() async {  
  final response = await http.get(  
    Uri.parse('https://api.example.com/students'),  
  );  
  
  if (response.statusCode == 200) {  
    List<dynamic> data = jsonDecode(response.body);  
    print(data);  
  } else {  
    print('Error: ${response.statusCode}');  
  }  
}
```

```
}
```

8.3 FutureBuilder (Pag-display ug Data Gikan sa API)

```
FutureBuilder<String>(  
  future: fetchData(),  
  builder: (context, snapshot) {  
    if (snapshot.connectionState == ConnectionState.waiting) {  
      return CircularProgressIndicator(); // loading spinner  
    } else if (snapshot.hasError) {  
      return Text('Error: ${snapshot.error}');  
    } else {  
      return Text('Result: ${snapshot.data}');  
    }  
  },  
)
```

PART 9: LISTS UG DATA DISPLAY

9.1 ListView.builder (Para sa dinamikong listahan)

```
List<String> students = ['Juan', 'Maria', 'Pedro'];  
  
ListView.builder(  
  itemCount: students.length,  
  itemBuilder: (context, index) {  
    return ListTile(  
      leading: Icon(Icons.person),  
      title: Text(students[index]),  
      onTap: () => print('Gi-tap si ${students[index]}'),  
    );  
  },  
)
```

Explanation: ang `.builder` constructor mas efficient kaysa basic `ListView` — mo-build lang siya sa mga items nga makita sa screen (lazy loading), dili tanan diretso.

9.2 Models (Structured Data)

```
class Student {  
  String name;  
  int age;  
  double gpa;  
  
  Student({required this.name, required this.age, required this.gpa});  
  
  // Para ma-convert gikan sa JSON  
  factory Student.fromJson(Map<String, dynamic> json) {  
    return Student(  
      name: json['name'],
```

```

        age: json['age'],
        gpa: json['gpa'].toDouble(),
    );
}
}

```

9.3 ListView gamit ang Model

```

List<Student> students = [
    Student(name: 'Juan', age: 20, gpa: 3.5),
    Student(name: 'Maria', age: 21, gpa: 3.8),
];

ListView.builder(
    itemCount: students.length,
    itemBuilder: (context, index) {
        final student = students[index];
        return ListTile(
            title: Text(student.name),
            subtitle: Text('GPA: ${student.gpa}'),
        );
    },
)

```

PART 10: LOCAL DATA STORAGE

10.1 SharedPreferences (Simple Key-Value Storage)

```

dependencies:
  shared_preferences: ^2.2.0

```

```

import 'package:shared_preferences/shared_preferences.dart';

// Pag-save
Future<void> saveName(String name) async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setString('username', name);
}

// Pag-kuha
Future<String?> getName() async {
    final prefs = await SharedPreferences.getInstance();
    return prefs.getString('username');
}

```

Gamiton ni sa simple nga settings, login state, preferences — dili para sa daghan/complex nga data.

10.2 SQLite (Local Database)

```
dependencies:  
  sqflite: ^2.3.0  
  path: ^1.8.3
```

Gamiton ni kung kinahanglan nimo ug tinuod nga local database (parehas sa MySQL pero naa sa device mismo) — halimbawa, offline nga to-do list app o notes app.

10.3 Firebase (Cloud Database)

Sa mas dagko nga apps, kasagaran mo-gamit ug **Cloud Firestore** o **Realtime Database** gikan sa Firebase — real-time sync, authentication, cloud storage, tanan naa na.

PART 11: THEMING UG STYLING

11.1 Custom Theme

```
MaterialApp(  
  theme: ThemeData(  
    primarySwatch: Colors.blue,  
    scaffoldBackgroundColor: Colors.white,  
    textTheme: TextTheme(  
      bodyLarge: TextStyle(fontSize: 18, color: Colors.black87),  
    ),  
    elevatedButtonTheme: ElevatedButtonThemeData(  
      style: ElevatedButton.styleFrom(  
        backgroundColor: Colors.blue,  
        foregroundColor: Colors.white,  
      ),  
    ),  
  ),  
  home: HomeScreen(),  
)
```

11.2 Dark Mode

```
MaterialApp(  
  theme: ThemeData.light(),  
  darkTheme: ThemeData.dark(),  
  themeMode: ThemeMode.system, // mo-sunod sa system setting sa device  
)
```

11.3 Custom Fonts

```
# pubspec.yaml  
flutter:  
  fonts:  
    - family: Poppins  
      fonts:
```

```
- asset: fonts/Poppins-Regular.ttf
- asset: fonts/Poppins-Bold.ttf
  weight: 700
```

PART 12: ANIMATIONS

12.1 Implicit Animations (Sayon)

```
AnimatedContainer(
  duration: Duration(milliseconds: 300),
  width: isExpanded ? 200 : 100,
  height: isExpanded ? 200 : 100,
  color: isExpanded ? Colors.blue : Colors.red,
)
```

Explanation: matag naa'y usab sa value (halimbawa width), automatic na mo-animate padulong sa bag-ong value — dili na kinahanglan i-code ang tibuok animation logic.

12.2 Explicit Animations (Mas Detalyado nga Control)

```
class FadeExample extends StatefulWidget {
  @override
  _FadeExampleState createState() => _FadeExampleState();
}

class _FadeExampleState extends State<FadeExample>
  with SingleTickerProviderStateMixin {
  late AnimationController _controller;
  late Animation<double> _animation;

  @override
  void initState() {
    super.initState();
    _controller = AnimationController(
      duration: Duration(seconds: 2),
      vsync: this,
    );
    _animation = Tween<double>(begin: 0, end: 1).animate(_controller);
    _controller.forward();
  }

  @override
  Widget build(BuildContext context) {
    return FadeTransition(
      opacity: _animation,
      child: Text('Fade in ko!'),
    );
  }

  @override
  void dispose() {
```

```

    _controller.dispose(); // importante gyud, para dili mag-memory leak
    super.dispose();
  }
}

```

PART 13: TESTING

13.1 Unit Tests

```

import 'package:flutter_test/flutter_test.dart';

int add(int a, int b) => a + b;

void main() {
  test('Ang add function dapat mo-sum sa duha ka numbers', () {
    expect(add(2, 3), 5);
  });
}

```

13.2 Widget Tests

```

testWidgets('Counter increments smoke test', (WidgetTester tester) async {
  await tester.pumpWidget(MyApp());

  expect(find.text('0'), findsOneWidget);

  await tester.tap(find.byIcon(Icons.add));
  await tester.pump();

  expect(find.text('1'), findsOneWidget);
});

```

PART 14: ARCHITECTURE PATTERNS (Pro Level)

14.1 Ngano Kinahanglan ug Architecture?

Sa gagmay nga app, pwede lang basta-basta i-code. Pero sa dagko/team projects, kinahanglan naa'y **clear structure** para dali ma-maintain, ma-test, ug ma-scale.

14.2 MVC / MVVM Pattern (Konsepto)

```

lib/
├── models/           -- data classes (Student, Product, atbp.)
├── views/            -- UI screens/widgets
├── viewmodels/       -- business logic, naga-connect sa models ug views
├── services/         -- API calls, database operations
└── widgets/          -- reusable UI components

```

14.3 Clean Architecture (Konsepto, Advanced)

```
lib/
├── data/
│   ├── models/
│   ├── repositories/
│   └── datasources/      -- API, local DB
├── domain/
│   ├── entities/
│   ├── repositories/    -- abstract/interface
│   └── usecases/
└── presentation/
    ├── screens/
    ├── widgets/
    └── bloc/ (o provider/riverpod)
```

Explanation: ang tumong sa Clean Architecture kay **separation of concerns** — ang UI dili direkta naga-connect sa database/API, naa’y “layers” nga naga-separate sa responsibilities. Mas lisod i-setup pero mas dali i-test ug i-maintain sa dugay nga panahon.

14.4 Dependency Injection

```
// Gamit ang get_it package
final getIt = GetIt.instance;

void setup() {
  getIt.registerSingleton<ApiService>(ApiService());
}

// Pag-gamit bisan asa
final apiService = getIt<ApiService>();
```

PART 15: PERFORMANCE OPTIMIZATION

15.1 Tips para Paspas ang App

- Gamiton ang const constructor kung dili mag-usab ang widget (const Text('Hello')) — makatabang ni sa performance kay dili na i-rebuild pag-usab
- Ayaw ibutang ang heavy logic/computation sa sulod sa build() method
- Gamiton ang ListView.builder imbes ListView basta daghan items (lazy loading)
- I-avoid ang unnecessary nested widgets/deep widget trees
- Gamiton ang Image.network nga naa’y caching (halimbawa: cached_network_image package)

15.2 DevTools

```
flutter run --profile # run sa profile mode para ma-check ang performance
```

Gamiton ang **Flutter DevTools** (built-in tool) para ma-check ang widget rebuilds, memory usage, ug frame rendering time.

PART 16: BUILDING UG DEPLOYMENT

16.1 Build para sa Android

```
flutter build apk --release      # single APK file
flutter build appbundle --release # para sa Google Play Store (.aab)
```

16.2 Build para sa iOS

```
flutter build ios --release
# Kinahanglan pa i-archive gamit ang Xcode para ma-submit sa App Store
```

16.3 Build para sa Web

```
flutter build web
```

16.4 Environment Variables / Flavors

Kasagaran naa'y lain-laing "flavors" (development, staging, production) para dili masagol ang API keys/configs sa testing ug sa live/production app.

PART 17: QUICK REFERENCE CHEAT SHEET

Widget Lifecycle (StatefulWidget)

createState() → initState() → build() → ... → dispose()

- initState() — mo-run usa ra, sa pagsugod (maayo para sa pag-fetch ug initial data)
- build() — mo-run kada naa'y update sa UI
- dispose() — mo-run pag-tangtang na sa widget (importante para i-clean up ang controllers, listeners)

Kasagarang Sayop sa Beginners

- Kalimtan ang setState() — dili mo-update ang UI
- Kalimtan i-dispose ang TextEditingController o AnimationController — mag-memory leak
- Sobra nga nested widgets — lisod na basahon ug ma-debug
- Paggamit ug Container kung sakto na unta ang SizedBox (mas lightweight ang SizedBox para sa spacing lang)
- Dili pag-check kung null bago gamiton ang value (importante ang null safety sa Dart)

Package Manager (pubspec.yaml)

```
dependencies:
  flutter:
    sdk: flutter
  http: ^1.1.0
  provider: ^6.1.0
```

```
flutter pub get      # i-download ang mga dependencies  
flutter pub upgrade  # i-update ang mga packages
```

STUDY ROADMAP

1. **Week 1** — Dart basics (Part 1) + Flutter setup, basic widgets (Parts 2-3)
2. **Week 2** — Layout system deeply (Part 4), Navigation (Part 5)
3. **Week 3** — State management: setState → Provider (Parts 6-7), practice heavily diri
4. **Week 4** — Async/API calls (Part 8), Lists ug Models (Part 9), Local storage (Part 10)
5. **Week 5** — Theming, Animations (Parts 11-12), Testing (Part 13)
6. **Week 6** — Architecture patterns, performance, deployment (Parts 14-16) + himo ug real project

Pinaka-maayong paagi mo-practice: himo dayon ug simple nga apps habang nag-aral — halimbawa, To-Do List app, Simple Calculator, o Student Info app. Ang Flutter mas dali ma-kat-unan kung naga-code ka mismo, dili lang pag-basa.